



Universidade de Brasília

Departamento de Ciência da Computação

Procedimentos

.



Resumindo:

Operandos RISC-V

Nome	Exemplo	Comentários
32 Registradores	s0-s11, t0-t6, zero, a0-a7, gp, fp, sp, ra, tp	Locais rápidos para dados. No RISC-V, os dados precisam estar em registradores para a realização de operações aritméticas. O registrador RISC-V zero é sempre igual a 0 (<i>hardwired</i>)



Resumindo:

Assembly do RISC-V				
Categoria	Instrução	Exemplo	Significado	Comentários
Aritmética	add	add s1,s2,s3	$s1 = s2 + s3$	Três operandos; dados nos registr.
	subtract	sub s1,s2,s3	$s1 = s2 - s3$	Três operandos; dados nos registr.
	add immediate	addi s1,s2,100	$s1 = s2 + 100$	Usada para somar constantes
Transferência de dados	load word	lw s1,100(s2)	$s1 = \text{Memória}[s2+100]$	Dados da memória para o registr.
	store word	sw s1,100(s2)	$\text{Memória}[s2 + 100] = s1$	Dados do reg para a memória
	load byte	lb s1,100(s2)	$s1 = \text{Memória}[s2 + 100]$	Byte da memória para registrador
	store byte	sb s1,100(s2)	$\text{Memória}[s2+100] = s1$	Byte de um registrador para memória
	load upper immed.	lui s1,100	$s1 = 100 * 2^{12}$	Carrega kte 20 bits mais altos
Desvio condicional	branch on equal	beq s1,s2,25	if ($s1 == s2$) PC = PC+100	Testa ==; desvio relativo ao PC
	branch not equal	bne s1,s2,25	if ($s1 != s2$) PC = PC+100	Testa !=; desvio relativo ao PC
	set on less than	slt s1,s2,s3	if ($s2 < s3$) $s1 = 1$; else $s1 = 0$	Compara menor que; usado com beq, bne
	set less than immediate	slti s1,s2,100	if ($s2 < 100$) $s1 = 1$; else $s1 = 0$	Compara menor que constante
Desvio incondicional	jump	jal x0, 100	PC = PC + 100	Salto incondicional
	jump & link reg	jalr ra,100(x1)	$ra = PC+4$, $PC=x1+100$	Para switch e retorno de função
	jump and link	jal ra, 100	$ra = PC + 4$, $PC=PC+100$	Para chamada de função



Instruções de Suporte a Procedimentos

- Passos em um procedimento
 - Colocar os parâmetros em um lugar onde o procedimento possa acessá-los
 - Transferir o controle para o procedimento
 - Adquirir recursos de armazenamento necessários para o procedimento
 - Realizar a tarefa desejada
 - Colocar o valor de retorno em um lugar onde o programa que o chamou possa acessá-lo
 - Retornar o controle para o ponto de origem



Instruções de Suporte a Procedimentos

- Qual o lugar mais rápido que pode armazenar dados em um computador?
- Registradores RISC-V
 - a0 - a1: parâmetros para os procedimentos e valores de retorno
 - a2 - a7: argumentos para funções;
 - ra: registrador de endereço de retorno ao ponto de origem (ra = return address)



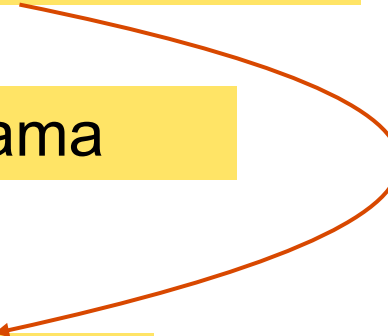
Instruções de suporte a procedimentos

Preparo argumentos para o procedimento

a0-a7



Desvio para procedimento



Continuação do programa

ra;
desvio



Execução do procedimento



Valores de retorno do procedimento

a0-a1



Retorno para a continuação do programa

jalr x0,0(ra)





JAL (jump and link)

- JAL x1, Label
 - Label é utilizado no montador. Localiza o endereço a ser alcançado pelo JAL
 - Na instrução JAL, o campo imm20 deve ser tal que :
 - $PC + imm20 * 2 = \text{Endereço}(\text{Label})$
- *Link*, neste caso, quer dizer que é armazenada, no registrador indicado (*x1 no exemplo*), o endereço da instrução que vem logo após a instrução *jal x1, Label*
- **x1** é denominado **ra**, **return address**, e é utilizado por padrão para armazenar o endereço de retorno de função
- Entretanto, qualquer outro registrador pode ser utilizado



Exemplo de procedimento

C

```
main()
{
    ...
    c=soma(a,b);... /* a=s0; b=s1; c=s2 */
    ...
}

int soma(int x, int y) /* x=a0; y=a1 */
{
    return x+y;
}
```

R
I
S
C
V

endereço

0x00400000	add a0, s0, zero	# x = a
0x00400004	add a1, s1, zero	# y = b
0x00400008	jal ra, soma	# prepara ra e j soma
0x0040000C	add s2, a0, zero	# s2=a+b
...		
0x00400100	soma: add a0, a0, a1	
0x00400104	jalr x0, 0(ra)	# retorna



Usando mais registradores

- Se precisar mais de 8 argumentos e 2 valores de retorno?
- Se o procedimento necessitar utilizar registradores salvos $\$x$?



Usando mais registradores

- Processo conhecido por *register spilling*
 - Uso de uma pilha;
 - Temos um apontador para o topo da pilha;
 - Este apontador é ajustado em uma palavra para cada registrador que é colocado na pilha (push), ou retirado da pilha (pop).
 - No RISC-V, o registrador **x2** é utilizado somente para indicar o topo da pilha: **sp (stack pointer)**



Usando a Pilha

- Por razões históricas, a pilha “cresce” do maior endereço para o menor endereço
- Para colocar um valor na pilha (*push*), devemos decrementar *sp* em uma palavra e mover o valor desejado para a posição de memória apontada por *sp*
- Para retirar um valor da pilha (*pop*), devemos ler este valor da posição de memória apontado por *sp*, e então incrementar *sp* em uma palavra



Exemplo

- Suponha que tenhamos o seguinte código:

```
int exemplo_folha (int g, int h, int i, int j)
{
    int f;
    f = (g+h) – (i+j);
    return f;
}
```

- Vamos gerar o código correspondente em assembly RV32I.



Exemplo-folha

- Definição: Os argumentos g , h , i e j correspondem aos registradores $a0$, $a1$, $a2$ e $a3$, e f corresponde a $s0$.
- Definir o rótulo do procedimento:
exemplo_folha:
- Devemos então armazenar na pilha os registradores que serão utilizados pelo procedimento:

```
addi sp, sp, -12    # cria espaço para 3 itens na pilha
sw t1, 8(sp)        # empilha t1
sw t0, 4(sp)        # empilha t0
sw s0, 0(sp)        # empilha s0
```

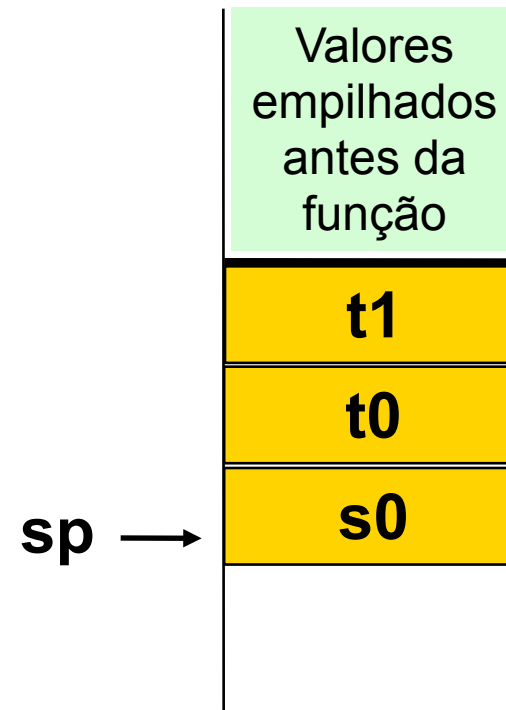


Exemplo-folha

Como ficou a pilha?



**Pilha antes
da função**



**Pilha durante
execução da função**



Exemplo-folha

- Corpo do procedimento:

```
add t0, a0, a1      # t0 = g + h
add t1, a2, a3      # t1 = i + j
sub s0, t0, t1      # f = s0 = (g+h) - (i+j)
```

- Resultado é armazenado no registrador a0:

```
add a0, s0, zero    # retorna f em a0
```



Exemplo-folha

- Antes de sair do procedimento, restaurar os valores dos registradores salvos na pilha:

```
lw s0, 0(sp)      # desempilha s0
lw t0, 4(sp)      # desempilha t0
lw t1, 8(sp)      # desempilha t1
addi sp, sp, 12    # remove 3 itens da pilha
```

- Voltar o fluxo do programa para a instrução seguinte ao ponto em que a função exemplo_folha foi chamada:

```
jalr x0, 0(ra)    # retorna
```



Versão Didática

exemplo_folha:

```
addi sp, sp, -12
sw t1, 8(sp)
sw t0, 4(sp)
sw s0, 0(sp)
add t0, a0, a1
add t1, a2, a3
sub s0, t0, t1
add a0, s0, zero
lw s0, 0(sp)
lw t0, 4(sp)
lw t1, 8(sp)
addi sp, sp, 12
jalr x0, 0(ra)
```

```
int exemplo_folha (int g, int j, int i, int h)
{
    int f;
    f = (g+h) – (i+j);
    return f;
}
```

```
# cria espaço para 3 words na pilha
# empilha t1
# empilha t2
# empilha s0
# t0 = g + h
# t1 = i + j
# f = s0 = (g+h) – (i+j)
# retorna f em a0
# desempilha s0
# desempilha t0
# desempilha t1
# remove 3 words da pilha
# retorna
```



Versão Otimizada

- Salvar o que realmente necessitar ser salvo
- Por convenção, os registradores **ti** não precisam ser preservados.
- Utilizar registradores **si** onde realmente forem necessários.
- Ponderar uso de registradores com análise de desempenho.

exemplo_folha:

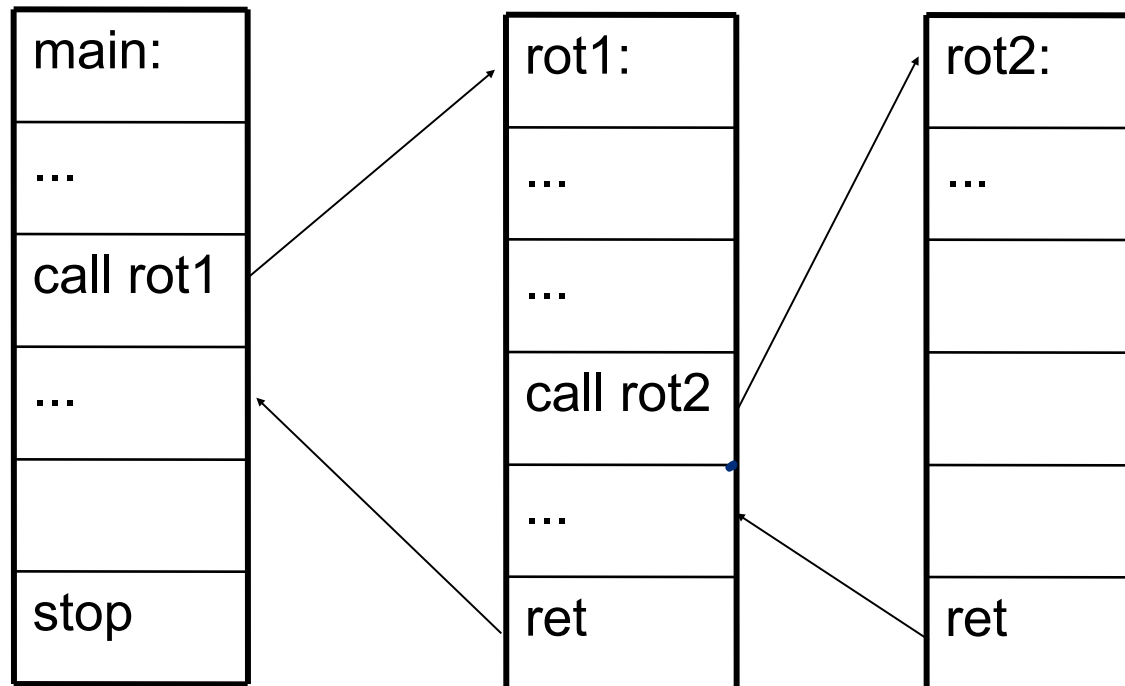
add a0, a0, a1	# a0 = g + h
sub a0, a0, a2	# a0 = g+h-i
sub a0, a0, a3	# f = a0 = g+h-i-j
ret	# jalr x0, 0(ra)



Procedimentos aninhados

- Suponha o seguinte procedimento aninhado:

Memória



- Problema: conflito com registradores **a** e **ra**



Procedimentos aninhados: convenção sobre registradores

- Uma solução é empilhar todos os registradores que precisam ser preservados
- Estabelecer uma convenção entre subrotinas chamada e chamadora sobre a preservação dos registradores (uso eficiente da pilha)
- Definições
 - **Chamadora:** função que faz a chamada, utilizando jal/call
 - **Chamada:** função sendo chamada



Por que utilizar convenções para chamadas de procedimentos?

■ **Benefícios:**

- programadores podem escrever funções que funcionam juntas;
- Funções que chamam outras funções – como as recursivas – funcionam corretamente.



Exemplo *strcpy*

- A função que copia strings em C pode ser descrita como segue:

```
void strcpy(char x[], char y[]) {  
    uint32_t i;  
    i = 0;  
    while ((x[i] = y[i]) != '\0')  
        i += 1;  
}
```



Traduzindo para Risc-V

$x \Rightarrow x10$, $y \Rightarrow x11$, $i \Rightarrow x19$

strcpy:

```
addi    sp, sp, -8    // salvar x19
sw      x19, 0(sp)
add     x19, x0, x0    // i = 0+0
```



Traduzindo para Risc-V

$x \Rightarrow x10, y \Rightarrow x11, i \Rightarrow x19$

strcpy:

```
        addi    sp, sp, -8    // salvar x19
        sw      x19, 0(sp)
        add     x19, x0, x0    // i = 0+0
L1:     add     x5, x19, x11    // x5 = &y[i]
        lbu     x6, 0(x5)      // x6 = y[i]
        add     x7, x19, x10    // x7 = &x[i]
        sb      x6, 0(x7)      // x[i] = y[i]
```



Traduzindo para Risc-V

$x \Rightarrow x10$, $y \Rightarrow x11$, $i \Rightarrow x19$

strcpy:

```
        addi    sp, sp, -8    // salvar x19
        sw      x19, 0(sp)
        add     x19, x0, x0    // i = 0+0
L1:     add     x5, x19, x11    // x5 = &y[i]
        lbu     x6, 0(x5)      // x6 = y[i]
        add     x7, x19, x10    // x7 = &x[i]
        sb      x6, 0(x7)      // x[i] = y[i]
        beq     x6, x0, L2
        addi    x19, x19, 1    // i = i + 1
        jal     x0, L1         // go to L1
```



Traduzindo para Risc-V

$x \Rightarrow x10$, $y \Rightarrow x11$, $i \Rightarrow x19$

strcpy:

```
        addi    sp, sp, -8    // salvar x19
        sw      x19, 0(sp)
        add     x19, x0, x0    // i = 0+0
L1:     add     x5, x19, x11    // x5 = &y[i]
        lbu     x6, 0(x5)      // x6 = y[i]
        add     x7, x19, x10    // x7 = &x[i]
        sb      x6, 0(x7)      // x[i] = y[i]
        beq     x6, x0, L2
        addi    x19, x19, 1    // i = i + 1
        jal     x0, L1         // go to L1
L2:     lw      x19, 0(sp)      // recupera x19
        addi    sp, sp, 8
        jalr    x0, 0(x1)      // retorna
```